

Publier le projet

Publier est **irréversible**. Un dépôt rendu public, même repassé en privé quelques minutes plus tard, a pu être cloné, indexé par un moteur de recherche ou archivé. On ne dépublie pas une donnée, on constate qu'elle a circulé.

Ce guide sert à décider **quoi** publier, et à vérifier ce qui part avant que cela parte.

1. À qui appartient ce projet

Le dépôt vit aujourd'hui sur un compte personnel. Le travail, lui, a été commandé par CHRUTH et porte sur son activité commerciale.

Ces deux faits ne se contredisent pas, mais ils ne donnent pas le même droit :

Élément	Qui décide de le publier
Le code que tu as écrit	Toi, sauf clause de cession dans ton contrat
Les données collectées (appels d'offres, acheteurs, scores)	CHRUTH
La base prospects (132 000 sociétés)	CHRUTH, et le RGPD s'y applique
Le nom, le logo, la fiche entreprise	CHRUTH

À faire avant toute publication de données : obtenir l'accord écrit de CHRUTH. Un accord oral suffit rarement quand la donnée est déjà en ligne.

2. Ce que publier expose réellement

Les avis d'appels d'offres sont publics : le BOAMP les diffuse. Ce n'est pas eux qui posent problème.

Ce qui est propre à CHRUTH, c'est **la sélection et le classement** : quels marchés sont jugés pertinents, avec quel score, quels acheteurs sont visés en priorité. Publier cela revient à publier la stratégie commerciale — un concurrent y lit la liste des marchés que CHRUTH prépare, et l'ordre dans lequel il compte les attaquer.

La base prospects est un cas plus lourd encore : 132 000 sociétés avec adresses et effectifs, enrichies depuis des sources publiques. Les rendre disponibles en un fichier téléchargeable est un traitement de données que le RGPD encadre. **Ne pas la publier.**

3. Trois choses différentes, trois décisions

On dit « publier le projet » pour trois gestes qui n'ont ni le même intérêt ni le même risque.

Chemin A — publier seulement l'état de la veille

Pourquoi : pour que `CHRUTH_PLATEFORME.html` lise des données fraîches sans qu'on relance le générateur. C'est le seul chemin qui apporte quelque chose que tu n'as pas déjà.

Ce qui sort : le fichier `etat/veille.json` — objets des marchés, acheteurs, dates, verdicts de tri. Pas la base prospects, pas le code, pas les réglages.

Comment :

1. Créer un dépôt **séparé** et public, par exemple `chruth-veille-publique`. Séparé, pour que la décision porte sur ce seul fichier et jamais sur le reste.
2. Y publier uniquement `etat/veille.json`, expurgé de son bloc `reglages` — il contient les adresses des destinataires.
3. L'URL de lecture est alors : `https://raw.githubusercontent.com/<compte>/chruth-veille-publique/main/veille.json`
4. Cette adresse autorise la lecture depuis une page web, ce qui permet au miroir HTML d'aller la chercher.

Ce qu'il faut accepter : n'importe qui disposant de l'URL lit la veille. Elle n'est pas secrète, elle est simplement peu susceptible d'être devinée — ce qui n'est pas une protection.

Si le dépôt doit rester privé : l'URL réclame alors un jeton, qu'il faudrait écrire dans le fichier HTML, donc le distribuer à qui l'ouvre. Cela revient à publier le jeton. Dans ce cas, renoncer au chemin A et garder la régénération par le script.

Chemin B — publier l'application

Pourquoi : consulter la veille depuis un téléphone, avec la vraie application et non un miroir.

C'est déjà documenté pas à pas : voir `docs/DEPLOIEMENT_APP_VEILLE.md`. L'application est protégeable par la page de connexion — voir `docs/GUIDE_CONNEXION.md` — ce qui la rend publiable sans exposer les données à tout le monde. **C'est le meilleur rapport entre l'utilité et le risque.**

Chemin C — publier le code

Pourquoi : montrer ton travail, ou permettre à un tiers de reprendre le projet.

À retirer avant : ce n'est pas le code qui pose problème, ce sont les données et les secrets qui l'accompagnent. Voir la liste au point 4.

À ajouter : un fichier de licence. Sans licence, personne n'a le droit de réutiliser le code — publier sans licence ne publie rien d'utilisable.

4. Ce qui ne doit jamais sortir

Fichier	Contenu	Déjà ignoré par git
<code>.env</code>	Clés des API d'IA	oui
<code>alertes_secrets.json</code>	Mot de passe d'application Gmail	oui
<code>destinataires.txt</code>	Adresses des destinataires	oui
<code>.streamlit/secrets.toml</code>	Identifiants de connexion, liste des accès	oui
<code>etat/veille.json</code>	Bloc <code>reglages</code> avec les adresses	oui
<code>data/</code>	Base et réponses brutes	oui
<code>output/prospects_*.csv</code>	132 502 sociétés — RGPD	non, voir ci-dessous

Vérifier que la règle tient toujours, plutôt que de la croire sur parole :

```
for f in .env alertes_secrets.json destinataires.txt .streamlit/secrets.toml; do
  git check-ignore -q "$f" && echo "IGNORE $f" || echo "EXPOSE $f"
done
```

Le point bloquant : `output/` est ignoré mais déjà suivi

`.gitignore` contient bien `output/` en ligne 3. **Cette règle n'a aucun effet sur les fichiers déjà commités**, et ceux-ci l'ont été avant qu'elle existe.

Constat au 29/07/2026, mesuré et non supposé :

Fichiers suivis dans <code>output/</code>	40
Volume	364 Mo
<code>prospects_enrichis.csv</code>	132 502 sociétés , avec <code>denomination</code> et <code>adresse_complete</code>
Également suivis	<code>prospects_nettoyes.csv</code> , <code>Base_Prospects_CHRUTH.xlsm</code> (81 Mo), <code>powerbi_sources/Prospects.csv</code> , <code>Carte_Prospects_CHRUTH.html</code>

Conséquence : rendre ce dépôt public publierait la base prospects immédiatement. Et comme elle est dans l'historique, la retirer aujourd'hui ne l'en sortirait pas — n'importe quel commit antérieur la contient encore.

Vérifier soi-même :

```
git ls-files output/ | wc -l  
git log --all --name-only --format="" | sort -u | grep -i prospects
```

Trois issues, par ordre de simplicité :

1. **Garder le dépôt privé.** Rien à faire, et c'est cohérent : ce dépôt contient les livrables d'un client, pas un projet à partager.
2. **Publier le code dans un dépôt neuf**, sans reprendre l'historique. On y copie les fichiers `.py`, les guides et les tests, on initialise un dépôt vierge, et la base prospects n'y entre jamais. C'est plus sûr et plus rapide que de réécrire 364 Mo d'historique.
3. **Réécrire l'historique** (`git filter-repo`) : long, risqué, et inutile si l'option 2 convient. À réserver au cas où l'historique lui-même doit être conservé.

Pour empêcher que cela s'aggrave, sans toucher au passé :

```
git rm --cached -r output/
```

Cela cesse de suivre ces fichiers à partir du prochain commit. Ils restent sur le disque, et restent dans les commits passés.

Vérifier l'historique

Un fichier **déjà commité** reste dans l'historique même après avoir été ajouté au `.gitignore`. Vérifier aussi le passé :

```
git log --all --name-only --format="" | sort -u | grep -E "\.env|secrets|destinataires"
```

Si un secret apparaît, il est compromis : le révoquer et en créer un nouveau. Le retirer de l'historique ne suffit pas, il a pu être lu.

5. Avant de pousser, dans cet ordre

1. **Accord de CHRUTH** sur ce qui est publié — par écrit.
2. **Vérifier les secrets** avec les deux commandes du point 4.
3. **Produire une copie propre** pour contrôler ce que voit un tiers : `bash python outils/preparer_dossier_demo.py` Le script refuse d'écrire s'il détecte un fichier de secrets, et masque les adresses recopiées ailleurs. Ouvrir la copie et la parcourir.
4. **Vérifier le miroir HTML** s'il est du lot — il embarque des données : `bash python outils/generer_plateforme_html.py --verifier`
5. **Pousser**, puis relire ce qui est en ligne depuis une fenêtre de navigation privée, déconnecté. C'est la seule façon de voir ce que voit un inconnu.

6. Ce que je recommande

Ne pas rendre ce dépôt public en l'état. Il porte la base prospects dans son historique : la publication serait immédiate et définitive. Ce point prime sur tous les autres.

Ensuite, le chemin B et lui seul pour commencer : déployer l'application avec la page de connexion activée. Tu obtiens l'accès mobile sans rien exposer, et la décision reste réversible — il suffit de retirer une adresse de la liste.

Le chemin A n'a d'intérêt que si tu veux un fichier autonome toujours frais, et il suppose d'accepter que la veille soit lisible par qui connaît l'URL. Il ne demande pas de rendre ce dépôt public : il passe par un dépôt séparé ne contenant qu'un fichier.

Le chemin C peut attendre. S'il devient nécessaire, passer par un dépôt neuf sans historique — jamais par une bascule en public de celui-ci.

7. Récapitulatif des vérifications

Contrôle	Commande	État au 29/07/2026
Secrets ignorés	<code>git check-ignore -q .env</code> etc.	5 fichiers sur 5 ignorés
Secrets dans l'historique	<code>git log --all --name-only --format=""</code>	aucun
Base prospects suivie	<code>git ls-files output/</code>	40 fichiers, 364 Mo — à traiter
Copie sans secret	<code>python outils/preparer_dossier_demo.py</code>	passé
Miroir HTML	<code>python outils/generer_plateforme_html.py --verifier</code>	passé

Aucun secret n'a jamais été commité, et les cinq fichiers sensibles sont bien ignorés. Le seul point ouvert est `output/`.